Q2.1: Add Staff ID Validation (BUSINESS LOGIC)

Question: Add validation to ensure Staff IDs are unique. Create an isStaffIDUnique(String staffID) method in WestminsterHotelManager and integrate it into the addHotelStaff() method to prevent duplicate IDs.

SOLUTION STEPS:

STEP 1: Add Validation Method to WestminsterHotelManager.java

ADD THIS METHOD ANYWHERE IN THE CLASS (before the closing }):

```
// Validation method to check if Staff ID is unique
public boolean isStaffIDUnique(String staffID) {
   for (HotelStaff staff : hotelStaffList) {
      if (staff.getStaffID().equals(staffID)) {
         return false; // ID already exists
      }
   }
   return true; // ID is unique
}
```

STEP 2: Update addHotelStaff() Method

FIND THIS SECTION:

```
System.out.println("Enter the staff ID");
String staffID = s.nextLine();
```

REPLACE WITH:

```
System.out.println("Enter the staff ID");
String staffID = s.nextLine();


// Validate Staff ID uniqueness
while (!isStaffIDUnique(staffID)) {
   System.out.println("Staff ID already exists! Please enter a unique staff ID:");
   staffID = s.nextLine();
}
```

Q2.2: Add Room Number Validation (BUSINESS LOGIC)

Question: Add room number validation to prevent double booking. Create an isRoomNumberAvailable(int roomNumber) method and use it in the addGuest() method to ensure no two guests are assigned the same room.

SOLUTION STEPS:

STEP 1: Add Validation Method to WestminsterHotelManager.java

```
// Validation method to check if room number is available
public boolean isRoomNumberAvailable(int roomNumber) {
   for (Guest guest : hotelGuestList) {
      if (guest.getRoomNumber() == roomNumber) {
         return false; // Room is occupied
      }
   }
   return true; // Room is available
}
```

STEP 2: Update addGuest() Method

FIND THIS SECTION:

```
System.out.println("Enter the room number");
int roomNum = s.nextInt();
s.nextLine();
```

REPLACE WITH:

```
System.out.println("Enter the room number");
int roomNum = s.nextInt();
s.nextLine();


// Validate room number availability
while (!isRoomNumberAvailable(roomNum)) {
   System.out.println("Room number " + roomNum + " is already occupied! Please enter a different room number:");
```

```
    roomNum = s.nextInt();
    s.nextLine();
}
```

Q2.3: Create Guest Management GUI (MAJOR GUI EXTENSION)

Question: Create a GuestTableGUI class and GuestTableModel class to display guests in a table format similar to the staff table. Add a new menu option (5) to open the Guest GUI with columns: Name, Surname, Room Number, Nights Stayed.

SOLUTION STEPS:

STEP 1: Create GuestTableModel.java (NEW FILE)

```
package com.mycompany.hotelmanagementsystem;

import java.util.ArrayList;
import javax.swing.table.AbstractTableModel;

public class GuestTableModel extends AbstractTableModel {

    private String[] columnNames = {"Name", "Surname", "Room Number", "Nights Stayed"};
    private ArrayList<Guest> list;

    public GuestTableModel(ArrayList<Guest> guestList) {
        list = guestList;
    }

    @Override
    public int getRowCount() {
        return list.size();
    }
```

```
    @Override
    public int getColumnCount() {
        return columnNames.length;
    }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        Object temp = null;
        if (columnIndex == 0) {
            temp = list.get(rowIndex).getName();
        } else if (columnIndex == 1) {
            temp = list.get(rowIndex).getSurname();
        } else if (columnIndex == 2) {
            temp = list.get(rowIndex).getRoomNumber();
        } else if (columnIndex == 3) {
            temp = list.get(rowIndex).getNightsStayed();
        }
        return temp;
    }

    @Override
    public String getColumnName(int col) {
        return columnNames[col];
    }
}
```

STEP 2: Create GuestTableGUI.java (NEW FILE)

```
package com.mycompany.hotelmanagementsystem;
```

```
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.util.ArrayList;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JScrollPane;
import javax.swing.JTable;

public class GuestTableGUI extends JFrame {

   JTable myTable;
   GuestTableModel tableModel;
   ArrayList<Guest> list;

   public GuestTableGUI(ArrayList<Guest> list) {
      this.setTitle("Guests in Hotel Management System");
      this.list = list;
      tableModel = new GuestTableModel(list);
      myTable = new JTable(tableModel);

      // Set the size of the frame
      setBounds(50, 50, 700, 500);
      setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // Close only this window

      // Enable sorting
      myTable.setAutoCreateRowSorter(true);
```

```
        // Add the table to the panel
        JScrollPane scrollPane = new JScrollPane(myTable);
        scrollPane.setPreferredSize(new Dimension(380, 280));

        // Add Guest Info button
        JButton infoButton = new JButton("Guest Info");
        infoButton.addActionListener(e -> {
            int selectedRow = myTable.getSelectedRow();
            if (selectedRow >= 0) {
                int modelRow = myTable.convertRowIndexToModel(selectedRow);
                Guest guest = list.get(modelRow);
                JOptionPane.showMessageDialog(this, guest.toString(), "Guest Information",
JOptionPane.INFORMATION_MESSAGE);
            } else {
                JOptionPane.showMessageDialog(this, "Please select a guest first!", "No Selection",
JOptionPane.WARNING_MESSAGE);
            }
        });

        // Add the panel to the frame
        add(scrollPane, BorderLayout.CENTER);
        add(infoButton, BorderLayout.SOUTH);
    }
}
```

STEP 3: Update WestminsterHotelManager.java - Add Menu Option

FIND THIS SECTION:

```
System.out.println("To Open GUI, press 4");
```

ADD AFTER IT:

```
System.out.println("To Open Guest GUI, press 5");
```

FIND THE SWITCH STATEMENT:

```
case 4:
  this.runGUI();
  break;
```

ADD AFTER IT:

```
case 5:
  this.runGuestGUI();
  break;
```

STEP 4: Add runGuestGUI() Method to WestminsterHotelManager.java

```
// Method to open Guest GUI
public void runGuestGUI() {
  GuestTableGUI guestTable = new GuestTableGUI(hotelGuestList);
  guestTable.setVisible(true);
}
```

Q2.4: Add Guest Info Button Enhancement (GUI EXTENSION)

Question: Add a "Guest Info" button to the guest table that displays complete guest information in a popup dialog when clicked.

SOLUTION:

This is already included in Q2.3 above, but here's the standalone solution:

Update GuestTableGUI.java Button Section:

```
// Add Guest Info button with functionality
JButton infoButton = new JButton("Guest Info");
infoButton.addActionListener(e -> {
  int selectedRow = myTable.getSelectedRow();
  if (selectedRow >= 0) {
    int modelRow = myTable.convertRowIndexToModel(selectedRow);
    Guest guest = list.get(modelRow);
```

```
    // Create detailed info string
    String guestInfo = "Guest Details:\n" +
            "Name: " + guest.getName() + " " + guest.getSurname() + "\n" +
            "Room Number: " + guest.getRoomNumber() + "\n" +
            "Nights Stayed: " + guest.getNightsStayed();

    JOptionPane.showMessageDialog(this, guestInfo, "Guest Information",
JOptionPane.INFORMATION_MESSAGE);
  } else {
    JOptionPane.showMessageDialog(this, "Please select a guest first!", "No Selection",
JOptionPane.WARNING_MESSAGE);
  }
});
```

Q2.5: Calculate Total Revenue Method (BUSINESS LOGIC)

Question: Add a calculateTotalRevenue() method to WestminsterHotelManager that calculates the total revenue from all guests. Assume each night costs £50. The method should return a double value.

SOLUTION STEPS:

STEP 1: Add Method to WestminsterHotelManager.java

```
// Business logic method to calculate total revenue
public double calculateTotalRevenue() {
  double totalRevenue = 0.0;
  double pricePerNight = 50.0; // £50 per night

  for (Guest guest : hotelGuestList) {
    totalRevenue += guest.getNightsStayed() * pricePerNight;
  }

  return totalRevenue;
```

```
}
```

STEP 2: Add Menu Option to Display Revenue

ADD TO MENU DISPLAY:

```
System.out.println("To View Total Revenue, press 6");
```

ADD TO SWITCH STATEMENT:

```
case 6:
   double revenue = this.calculateTotalRevenue();
   System.out.printf("Total Hotel Revenue: £%.2f%n", revenue);
   break;
```

STEP 3: Alternative - Add to Hotel Statistics Method

```
// Enhanced method that shows revenue with other stats
public void printHotelStatistics() {
   System.out.println("\n=== HOTEL STATISTICS ===");
   System.out.println("Total Staff: " + hotelStaffList.size());
   System.out.println("Total Guests: " + hotelGuestList.size());
   System.out.printf("Total Revenue: £%.2f%n", calculateTotalRevenue());

   // Calculate average nights per guest
   if (!hotelGuestList.isEmpty()) {
      double totalNights = 0;
      for (Guest guest : hotelGuestList) {
         totalNights += guest.getNightsStayed();
      }
      double averageNights = totalNights / hotelGuestList.size();
      System.out.printf("Average Nights per Guest: %.1f%n", averageNights);
   }
}
```

Q2.6: Count Staff by Type Method (BUSINESS LOGIC)

Question: Add a getStaffCountByType(String staffType) method that returns the number of staff members of a specific type (Manager, HouseKeeper, etc.).

SOLUTION STEPS:

STEP 1: Add Method to WestminsterHotelManager.java

```
// Business logic method to count staff by type
public int getStaffCountByType(String staffType) {
    int count = 0;

    for (HotelStaff staff : hotelStaffList) {
        if (staffType.equals("Manager") && staff instanceof Manager) {
            count++;
        } else if (staffType.equals("HouseKeeper") && staff instanceof HouseKeeper) {
            count++;
        } else if (staffType.equals("Receptionist") && staff instanceof Receptionist) {
            count++;
        } else if (staffType.equals("Chef") && staff instanceof Chef) {
            count++;
        } else if (staffType.equals("SecurityGuard") && staff instanceof SecurityGuard) {
            count++;
        }
    }

    return count;
}
```

STEP 2: Enhanced Statistics Method

```
// Method to display comprehensive staff statistics
public void printStaffStatistics() {
    System.out.println("\n=== STAFF STATISTICS ===");
```

```
    System.out.println("Total Staff: " + hotelStaffList.size());
    System.out.println("Managers: " + getStaffCountByType("Manager"));
    System.out.println("HouseKeepers: " + getStaffCountByType("HouseKeeper"));
    System.out.println("Receptionists: " + getStaffCountByType("Receptionist"));
    System.out.println("Chefs: " + getStaffCountByType("Chef"));
    System.out.println("Security Guards: " + getStaffCountByType("SecurityGuard"));
}
```

STEP 3: Add Menu Option

```
// Add to menu
System.out.println("To View Staff Statistics, press 7");

// Add to switch
case 7:
    this.printStaffStatistics();
    break;
```

Q2.7: Extend Table with Details Column (TABLE EXTENSION)

Question: Add a fifth column "Details" to the staff table that shows specific information for each staff type: license number for managers, years of experience for housekeepers, and languages for receptionists.

SOLUTION STEPS:

STEP 1: Update HotelTableModel.java - Column Names

FIND THIS LINE:

```
private String[] columnNames = {"Name", "Surname", "Date of Birth", "Role"};
```

REPLACE WITH:

```
private String[] columnNames = {"Name", "Surname", "Date of Birth", "Role", "Details"};
```

STEP 2: Update getColumnCount() Method

No change needed - it automatically uses columnNames.length

STEP 3: Update getValueAt() Method

FIND THE EXISTING COLUMN 3 SECTION:

```
} else if (columnIndex == 3) {
   if (list.get(rowIndex) instanceof Manager) {
      temp = "Manager";
   } else if (list.get(rowIndex) instanceof HouseKeeper) {
      temp = "HouseKeeper";
   }
}
```

ADD AFTER IT:

```
} else if (columnIndex == 4) {
   // Details column - show specific info for each staff type
   if (list.get(rowIndex) instanceof Manager) {
      Manager manager = (Manager) list.get(rowIndex);
      temp = "License: " + manager.getLicenseNumber();
   } else if (list.get(rowIndex) instanceof HouseKeeper) {
      HouseKeeper housekeeper = (HouseKeeper) list.get(rowIndex);
      temp = "Experience: " + housekeeper.getYearsOfExperience() + " years";
   } else if (list.get(rowIndex) instanceof Receptionist) {
      Receptionist receptionist = (Receptionist) list.get(rowIndex);
      temp = "Languages: " + receptionist.getLanguagesSpoken();
   } else if (list.get(rowIndex) instanceof Chef) {
      Chef chef = (Chef) list.get(rowIndex);
      temp = "Speciality: " + chef.getSpeciality();
   } else if (list.get(rowIndex) instanceof SecurityGuard) {
      SecurityGuard guard = (SecurityGuard) list.get(rowIndex);
      temp = "Shift: " + guard.getShiftTime();
   }
}
```

📋 QUICK REFERENCE PATTERNS FOR TIER 2:

Validation Method Pattern:

```
public boolean isFieldValid(Type field) {
   for (ObjectType object : list) {
      if (object.getField().equals(field)) {
         return false; // Already exists
      }
   }
   return true; // Valid/Unique
}
```

Business Logic Method Pattern:

```
public ReturnType calculateSomething() {
   ReturnType result = initialValue;
   for (ObjectType object : list) {
      result += object.getSomeValue();
   }
   return result;
}
```

GUI Table Model Extension Pattern:

```
// Add column to columnNames array
private String[] columnNames = {"Col1", "Col2", "NewCol"};


// Add case to getValueAt()
} else if (columnIndex == X) {
   if (object instanceof Type1) {
      temp = ((Type1) object).getSpecificField();
   } else if (object instanceof Type2) {
```

```
        temp = ((Type2) object).getSpecificField();
    }
}
```

Count by Type Pattern:

```
public int getCountByType(String type) {
    int count = 0;
    for (ObjectType obj : list) {
        if (type.equals("TypeName") && obj instanceof TypeName) {
            count++;
        }
    }
    return count;
}
```

These 7 questions represent advanced features that demonstrate strong OOP and GUI programming skills!